Inference Optimisation for Large Models

Serving, Distillation, Pruning, Quantization, and TurboQuant

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

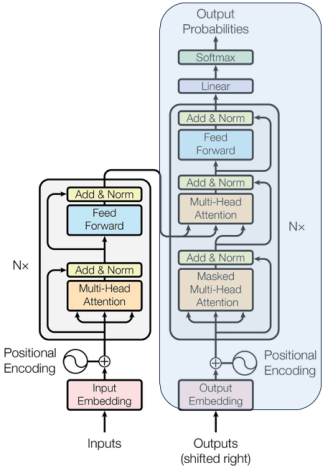
KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026

Table of Contents

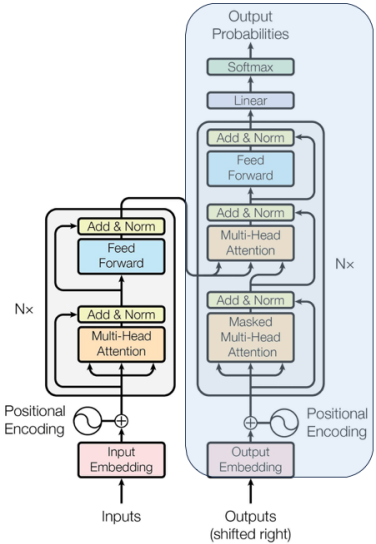
Understanding LLM Inference

- Most popular decoder-only LLMs (e.g., GPT-3) are pretrained for **causal language modeling**: predicting the next token in a sequence.
- At inference, LLMs receive a series of **tokens** as input and generate the following tokens **autoregressively** until a stopping criterion is met (e.g., token limit, stop word, or special <end> token).
- **Tokens** are about 4 English characters each.
- The inference process has **two phases**: **prefill** (process prompt in parallel) and **decode** (step-by-step token generation).



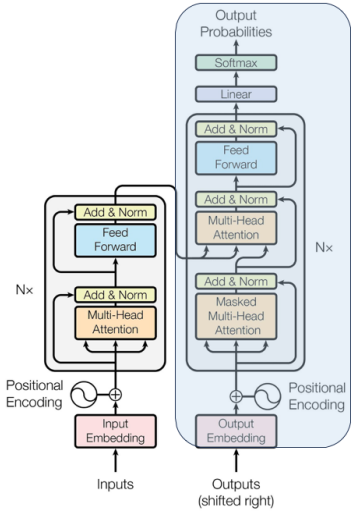
Prefill Phase: Parallel Input Processing

- **Prefill phase (input processing):**
  - All input tokens are processed together to compute **keys and values** (attention states) for the prompt.
  - The entire input sequence is known in advance, enabling a highly parallel **matrix-matrix** computation that fully leverages the GPU's throughput.
- **Token throughput caveat:**
  - Different LLMs may use different **tokenizers**, so "tokens per second" is not always comparable across models, as token-to-character mappings vary.



Decode Phase: Serial Token Generation

- **Decode phase (generating outputs):**
  - Output tokens are generated **one at a time**, each conditioned on all previously generated tokens.
  - Computation now becomes a **matrix-vector** operation, less parallelizable and typically **memory-bound**: moving weights/activations dominates speed, not raw compute.
  - Optimising decode is crucial: efficient attention, key/value caching, and memory management are ongoing research topics.



Why Optimise Inference?

- **Training** is often one-off; **inference** is repeated at scale (cost, latency, energy).
- Large language models are **memory-bound** and **autoregressive**: each new token revisits all previous tokens.
- Goals: lower **latency** (interactive UX), higher **throughput** (serving many users), smaller **memory** (long contexts, edge).

Measuring Inference

- **Time to first token (TTFT):** how long until the first output token appears; set by the prefill phase, which is compute-bound.
- **Time per output token (TPOT):** the gap between subsequent tokens; set by the decode phase, which is memory-bound.
- Request latency $\approx$ TTFT + TPOT $\times$ number of generated tokens.
- **Throughput:** total tokens per second across all requests. Batching raises throughput but can hurt per-request latency; serving systems balance the two.

Inference Optimisation: Key-Value Caching

- In **decode**, the model emits **one token per step**, but that token still depends on the **key and value** tensors for **all** earlier positions.
- Those positions include the **prompt** (K/V from **prefill**) plus any **new** K/V computed up to the current step.
- **KV caching** keeps these tensors in GPU memory so they are **not recomputed** at every step; each iteration **appends** the newly computed K/V to the cache for the next step.
- In many stacks there is **one KV cache per transformer layer**.

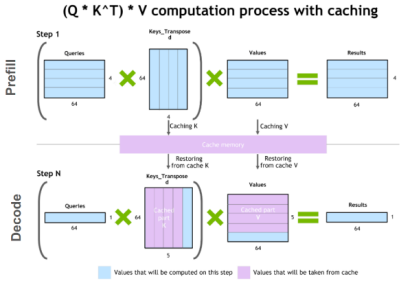
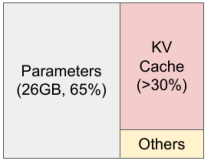


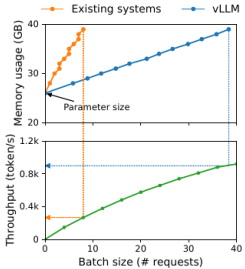
Figure: illustration of the key-value caching mechanism.

Reference: Verma & Vaidya, "Mastering LLM Techniques: Inference Optimization," NVIDIA Technical Blog (2023).

KV Cache Memory

- With batching, each request needs its own KV cache, which can become a bottleneck even before model weights do.
- Cost increases in proportion to batch size and sequence length, limiting throughput and ability to handle long inputs. This drives use of techniques like MQA/GQA, paging (PagedAttention), and KV quantization.





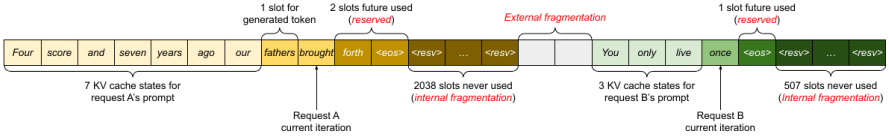
Serving a 13B model on an NVIDIA A100 (40 GB): weights take 65% of memory, the KV cache over 30%. Kwon et al., arXiv:2309.06180, Figure 1.

Batching: Static vs. Continuous

- A single decode stream is a matrix-vector workload that leaves most of the GPU idle; batching many requests turns it back into matrix-matrix work.
- **Static batching:** requests are grouped up front and the whole batch runs until its longest member finishes; short requests hold their slot while doing nothing.
- **Continuous batching:** scheduling happens per decode step; finished sequences leave the batch immediately and waiting requests join mid-flight (Yu et al., Orca, 2022).
- Standard in modern serving stacks (vLLM, TensorRT-LLM, TGI): large throughput gains with no change to model outputs.

KV Cache Fragmentation

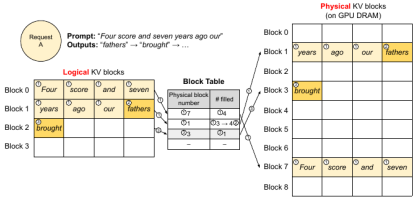
- Early serving systems stored each request's KV cache as one contiguous tensor, allocated up front for the maximum possible sequence length.
- Three kinds of waste follow: reserved slots not yet used, internal fragmentation (slots never used because the request ends early), and external fragmentation between buffers.
- Measured effective KV memory utilisation in such systems can drop to about 20%.



Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," arXiv:2309.06180, Figure 3.

PagedAttention (vLLM)

- Borrow virtual memory from operating systems: split each request's KV cache into fixed-size blocks and allocate blocks on demand.
- A block table maps logical positions to physical blocks, so the cache no longer needs to be contiguous; waste is limited to the last partially filled block, and blocks can be shared across sequences (parallel sampling, shared prefixes).
- vLLM built on this achieves 2–4$\times$ the throughput of prior serving systems at the same latency.

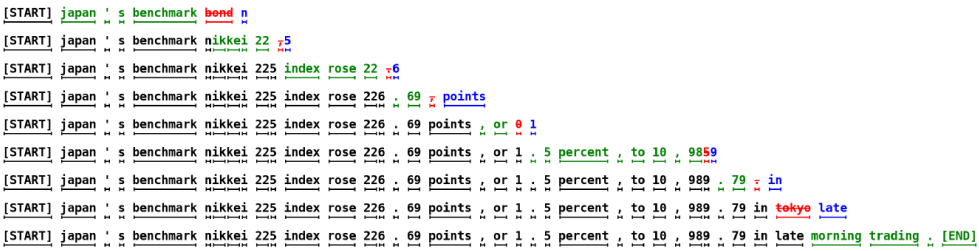


Block table translation. Kwon et al., arXiv:2309.06180, Figure 6.

Speculative Decoding: The Idea

- Decode is memory-bound: for the large model, scoring K proposed tokens in one forward pass costs about the same as generating a single token.
- A small **draft model** proposes K tokens autoregressively; the large **target model** then evaluates all K positions in parallel.
- Accept draft tokens while they are consistent with the target distribution; at the first rejection, resample that token from an adjusted distribution and discard the rest.
- The output distribution is provably identical to decoding with the target model alone; the speed comes from accepting several tokens per target pass.
- Reported speedups of 2–3$\times$ with no quality change (Leviathan et al., 2023; Chen et al., 2023).

Speculative Decoding: Acceptance in Practice



- Each line is one iteration: green tokens were proposed by the draft model and accepted, red were rejected, and blue are the target model's corrections.
- This 38-token sentence needed only 9 serial passes of the target model.
- Gains are largest when the draft agrees with the target on the easy tokens, which is most of them.

Leviathan, Kalman & Matias, "Fast Inference from Transformers via Speculative Decoding," arXiv:2211.17192, Figure 1.

Other Inference Levers

- **Kernel fusion & IO-aware attention** (e.g. FlashAttention-style): reduce HBM traffic.
- **Prefix caching:** reuse the KV cache of shared prompt prefixes (system prompts, few-shot examples) across requests.
- **Weight sharing / low-rank adapters (LoRA):** smaller footprints where applicable.
- **Approximate attention** (long contexts): trade accuracy for sub-quadratic cost.

Model Optimisation Techniques

Distillation

Harvard IACS AC215, Lecture 11 — Slides.

Compression Technique: Distillation

Problem:

- During training, a model does not have to operate in real time and does not necessarily face restrictions on computational resources. The primary goal is to extract as much structure from the given data as possible.
- But latency and resource consumption do become of concern if the model is to be deployed for inference.

So what? We must develop ways to compress models for inference.

Compression Technique: Distillation

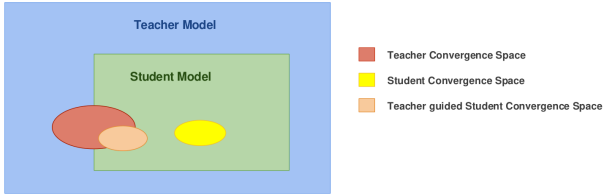
Idea:

- In 2006, Bucilă et al. showed that it was possible to transfer knowledge from a large trained model (or ensemble of models) to a smaller model for deployment by **training it to mimic the larger model's output**.
- In 2014 Hinton et al. generalized the process and gave the name **Distillation**.

Main idea of distillation is that **training and inference are 2 different tasks**; thus a **different model should be used**.

Distillation: Teacher – Student

Assumption: If we can achieve similar convergence using a **smaller network**, then the convergence space of the Teacher Network should **overlap** with the solution space of the Student Network.



Teacher/Student convergence regions. Harvard AC215, L11.

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

`This is a Dog` → one-hot → $\begin{matrix}0\\0\\1\\0\end{matrix}$ ⟶ Dog

Distillation: Soft Targets

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

`This is also a Dog`

one-hot $\begin{matrix}0\\0\\1\\0\end{matrix}$ $\longrightarrow$ Dog

Distillation: Soft Targets

While training, our confidence in the labels must be high.

Otherwise the data would not be present in the training set.

For example:

`This is a Cat` $\xrightarrow{\text{one-hot}}$ $\begin{matrix}0\\0\\1\\0\end{matrix}$ $\longrightarrow$ Dog

Distillation: Soft targets

What about this example?

`This is a ...`

$$\begin{pmatrix} 0 \\ 0.5 \\ 0.5 \\ 0 \end{pmatrix}$$

Distillation: Soft targets

In the real world, the distinction is never that clear. Ideally, the training labels must be soft, to learn about the nuanced relations across the elements of the dataset.

Distillation: Teacher – Student

The teacher shows the student the relations between the different classes, based on what it learned during its training stage.

From a dataset containing the original categories, the teacher outputs the normalized probabilities for each example.

	cow	car	dog	cat
Original labels	0	1	0	0
Probabilities	10^{-6}	0.9	0.1	10^{-9}

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*

Distillation: Teacher – Student (Softening with Temperature)

To enhance the student's learning, the teacher softens its output z even more.

To do this, it introduces the softmax function with temperature T:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

"Soft" targets allow the student to learn the teacher's implicit knowledge about class similarities.

	cow	car	dog	cat
Original labels	0	1	0	0
Softmax probabilities	10^{-6}	0.9	0.1	10^{-9}
Softened probabilities	0.05	0.3	0.2	0.005

Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean,

Distilling the Knowledge in a Neural Network

Distillation: Teacher – Student (Softening with Temperature)

A high value of T will dilute the information contained in the probabilities.

- The standard softmax is recovered for $T=1$.
- For $T>1$, the probabilities are softened. Typical values range between 2 and 5.
- For $T<1$, it approaches to the original one-hot labels.

	cow	car	dog	cat
Original labels	0	1	0	0
Softmax probabilities	10^{-6}	0.9	0.1	10^{-9}
Softened probabilities	0.05	0.3	0.2	0.005

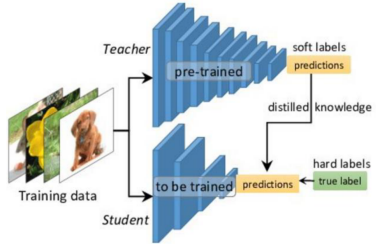
Adapted from:

Geoffrey Hinton, Oriol Vinyals & Jeff Dean,

Distilling the Knowledge in a Neural Network

Distillation: Teacher–Student Training

- The student is trained to both:
  - Solve the original task (matching the hard/true labels).
  - Replicate the softened probabilities (soft labels) from the teacher.
- This allows the student to learn not only the correct answer, but also the teacher's representation of class similarities.



Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*

Distillation: Teacher–Student Scenarios

Scenario 1:

Train on the same data as the teacher model.
(DistilBERT was trained on this example.)

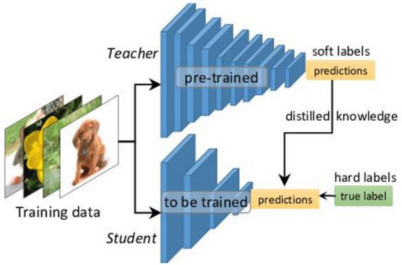
Scenario 2:

Train on a reduced dataset. This is usually done when you have the teacher model but not the original dataset or resources.

Distillation: Teacher–Student Loss

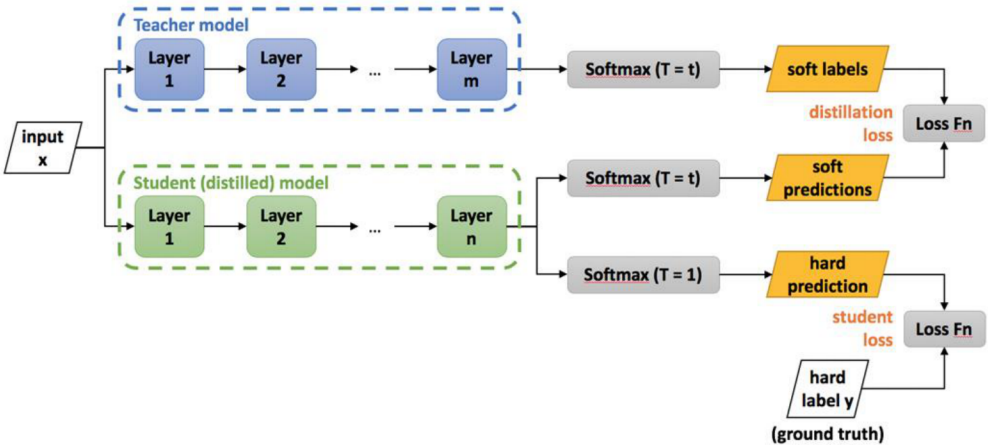
It minimizes the sum of two different cross-entropy functions:

- One involving the original hard labels obtained using a softmax with $T = 1$.
- One involving the softened probabilities with $T > 1$.



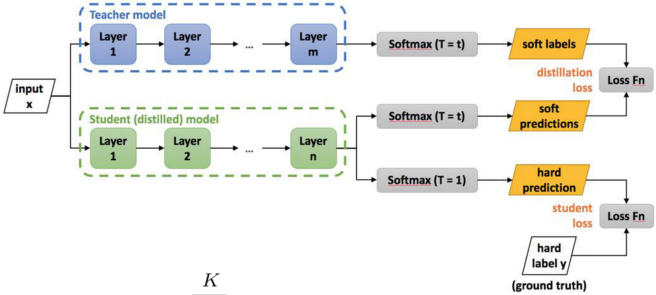
Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*

Distillation: Teacher–Student



Reference: Geoffrey Hinton, Oriol Vinyals & Jeff Dean, *Distilling the Knowledge in a Neural Network*.

Distillation: Teacher Student Training



$$L_{Distillation} = -\sum_{k}^{K} p_T^k(x,T) \log\left(q_S^k(x,T)\right)$$

33

Distillation: Teacher–Student

$$L_{\text{Distillation}} = -\sum_{k=1}^{K} p_T^k(x, T) \log\left(q_S^k(x, T)\right)$$

In our example, for the example n in the dataset:

$$p_T(x, T) = [0.05,\ 0.3,\ 0.2,\ 0.005]$$

And our predictions

$$q_S(x, T) = [0.1,\ 0.45,\ 0.30,\ 0.15]$$

The resulting loss value for this example is:

$$-\Big[0.05 \cdot \log(0.1) \ + \ 0.3 \cdot \log(0.45) \ + \ 0.2 \cdot \log(0.30) \ + \ 0.005 \cdot \log(0.15)\Big]$$

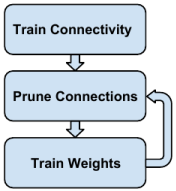
Distilling Reasoning Traces

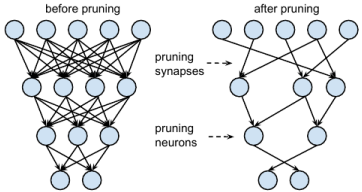
- Classic distillation transfers the teacher's **output distribution**: the student matches softened logits, one prediction at a time.
- For reasoning models the valuable signal is the **trace**: the chain of intermediate steps the teacher produces before its answer.
- **Recipe:** generate many problems' worth of full reasoning traces with a large reasoning model, filter for correct answers, then fine-tune the small model on the traces with plain supervised learning. No logits, no RL.
- DeepSeek distilled R1 traces into Qwen and Llama models from 1.5B to 70B this way; the distilled students beat much larger non-reasoning baselines on math and code benchmarks.
- The trade: trace distillation is far cheaper than running RL on the small model, but the student inherits the teacher's reasoning style, including its mistakes.

Pruning

Compression Technique: Pruning

- Trained networks are heavily over-parameterised: many weights contribute almost nothing to the output.
- **Magnitude pruning:** remove the connections with the smallest absolute weights, then retrain the survivors to recover accuracy; repeat iteratively.
- Han et al. (2015) cut AlexNet weights by 9× and VGG-16 by 13× with no accuracy loss.





Han et al., "Learning both Weights and Connections for Efficient Neural Networks," arXiv:1506.02626, Figures 2–3.

Pruning: Structure Matters

- **Unstructured pruning** removes individual weights: highest compression, but the result is a sparse matrix that standard GPU kernels cannot run faster.
- **Structured pruning** removes whole neurons, attention heads, or layers: less aggressive, but leaves a smaller dense model that is faster on any hardware.
- **2:4 semi-structured sparsity:** keep 2 weights in every group of 4; NVIDIA tensor cores (Ampere onward) run it at up to 2× math throughput (Mishra et al., 2021).
- For LLMs, one-shot post-training methods (SparseGPT, Wanda) reach about 50% sparsity with modest quality loss and no retraining.
- Pruning composes with distillation and quantization; production LLM serving still leans mostly on quantization, with pruning gaining ground through 2:4 hardware support.

Quantization

Quantization

Quantization is a **key** compression technique for **LLMs** and **large transformers** to fit on **constrained hardware**.

- **Goal:** run larger models on **constrained** hardware with acceptable accuracy.
- Lowering precision on **weights and/or activations** (e.g. FP32/FP16 $\rightarrow$ FP8) cuts **memory**, speeds **inference**, and reduces **energy**, at some accuracy cost; the right trade-off is use-case dependent.

What to Quantize

- **Weights:** straightforward win on footprint; Llama 2 7B at FP16/BF16 $\approx$ **14 GB** $\rightarrow$ FP8 weights $\approx$ **7 GB** (blog numbers).
- **Activations:** intermediate layer outputs; quantizing compute can exploit **tensor cores** at lower bit width for more throughput.
- **KV cache** (decoder-only): grows with context, layers, and heads, often **several GB** at long context (e.g. 4096 tokens) alongside weights.
- **Deployment patterns:** weight-only INT8/INT4 for maximum weight memory savings (activations may stay higher precision); **KV cache quantization** when long context dominates.

Quantization Data Types

- Floating point formats use their bits to store three parts: **sign** (positive/negative), **exponent** (size/range), and **mantissa** (precision of the value). For example, FP8 E4M3 uses 4 bits for the exponent and 3 bits for the mantissa.
- These numbers are represented as $x = (-1)^{\text{sign}} \cdot 2^{\text{exponent}} \cdot \text{mantissa}$.
- Typical formats: FP32, FP16, BF16, FP8, and FP4. Main difference is how much range and precision you get: higher precision means more accurate numbers, but also more memory and compute.

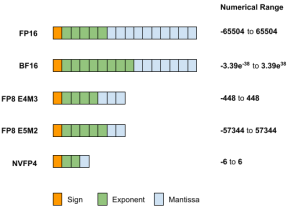


Figure: FP16, BF16, FP8, FP4 layouts.

Quantization Granularity

- **Per-tensor / per-layer:** one (s, z) for the whole tensor: simplest, smallest metadata, can mis-handle heterogeneous ranges.
- **Per-channel:** separate parameters per channel (e.g. conv channels): isolates **outliers** to their channel.
- **Per-block / per-group:** parameters per sub-block: finer control when mass varies inside a tensor.

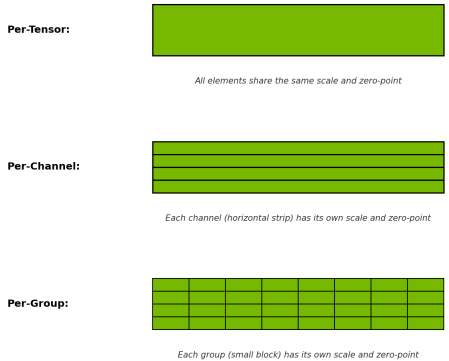


Figure: per-tensor, per-channel, per-block.

Quantization-Aware Training (QAT)

- Bake quantization into training: forward/backward see **fake-quant** (quantize then dequantize) while master weights stay high precision for optimiser steps.
- Weights adapt to **rounding/clipping** error; fake-quant modules often **frozen** while weights are fine-tuned.

PTQ: Weight-Only and Weight+Activation Quantization (Overview)

- **Weight-only quantization:** Quantize the model's trained weights to lower precision using a calculated scale (with or without a zero-point). Since weights are fixed after training, no input data is needed.
- **Weights and activations quantization:** Requires representative input data to measure activation statistics (by running the model), which are then used to determine quantization parameters (scale and zero-point) through a process called calibration.

PTQ: Static vs. Dynamic Quantization

- **Static quantization:** Calibration uses a dataset to determine quantization parameters once; these are kept fixed for all future inference.
- **Dynamic quantization:** Quantization parameters are computed on the fly during inference for each input, so no calibration set is needed but more runtime overhead is incurred.

TurboQuant

TurboQuant: Problem and Motivation

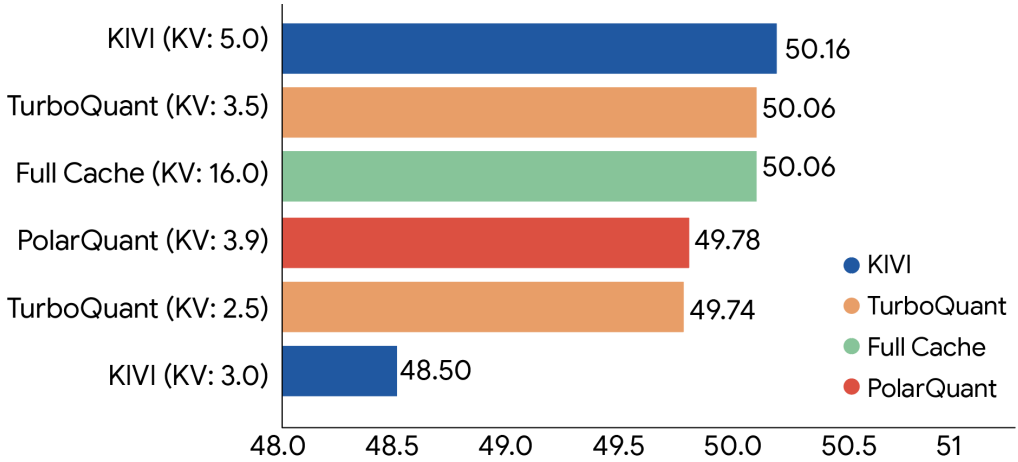
- **Vector quantization (VQ):** compress high-dimensional vectors while controlling **distortion** of their geometry.
  - Enhances vector search and KV cache compression.
- Prior practical schemes often fall **short of optimal** distortion–rate scaling.
- TurboQuant is a compression method that reduces model size with near-zero accuracy loss, addressing the constraints of long contexts and agentic workflows.
- Applies to any model at inference time with no offline preparation or calibration data.

TurboQuant: How It Works

- TurboQuant works in **two main stages:**
  - **Stage 1: High-quality compression with PolarQuant.** TurboQuant first randomly rotates the data vectors, simplifying their geometry. This lets us apply a strong, standard quantizer (like those used in audio or image compression) to each dimension, using most of the available bits to efficiently capture the main "strength" and "direction" of the original vector.
  - **Stage 2: Eliminating hidden errors with QJL.** After the main compression, a tiny bit of error remains. TurboQuant uses just 1 bit and the **QJL (Quantized Johnson–Lindenstrauss)** algorithm to compress this residual, acting like a mathematical error-checker that eliminates bias and ensures precise attention scores.

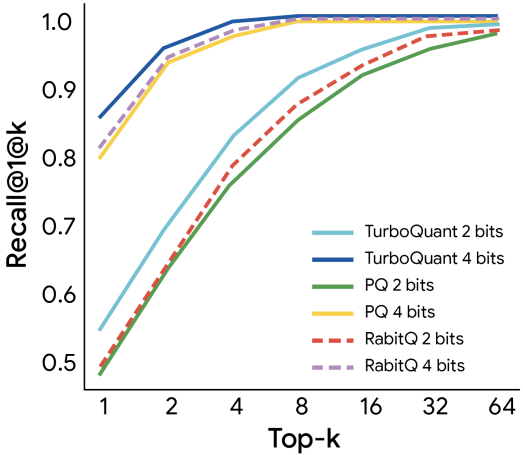
Zandieh & Mirrokni, "TurboQuant: Redefining AI efficiency with extreme compression," Google Research Blog (2026). Article.

TurboQuant: Experiments and Results



Aggregated scores across LongBench-style tasks (QA, code, summarization) for TurboQuant, PolarQuant, and KIVI baselines; bit-widths in brackets. Reference: Google Research Blog.

TurboQuant: Experiments and Results



1@k recall on GloVe (d=200): TurboQuant vs. PQ / RabitQ-style baselines. Reference: Google Research Blog.

Summary

- **Inference optimisation lowers latency, increases throughput, and reduces memory.**
- **KV caching** avoids recomputing past keys/values but grows memory with layers, batch, and length, motivating KV quantization, grouped attention, and paging.
- **Serving:** continuous batching keeps the GPU full; PagedAttention removes KV cache fragmentation; speculative decoding buys 2–3$\times$ decode speed with identical outputs.
- **Distillation:** train a compact **student** to mimic a **teacher** for deployment; **soft targets** at raised **temperature** carry knowledge beyond one-hot labels.
- **Pruning:** remove low-value weights (unstructured, structured, or 2:4 semi-structured) and retrain; composes with the other compression techniques.
- **Quantization:** shrink **weights**, **activations**, and the **KV cache** via lower-bit formats.
- **TurboQuant:** a compression method that reduces model size with near-zero accuracy loss, addressing long-context and agentic workflow constraints, and applies to any model at inference time with no offline preparation or calibration data.

References

1. Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. arXiv:1706.03762
2. Hinton, Vinyals, Dean, “Distilling the Knowledge in a Neural Network,” NIPS Deep Learning Workshop 2015. arXiv:1503.02531
3. Zelikman et al., “STaR: Bootstrapping Reasoning With Reasoning,” NeurIPS 2022. arXiv:2203.14465
4. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” CVPR 2018. arXiv:1712.05877
5. Leviathan, Kalman, Matias, “Fast Inference from Transformers via Speculative Decoding,” ICML 2023. arXiv:2211.17192
6. Chen et al., “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. arXiv:2302.01318
7. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP 2023. arXiv:2309.06180
8. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” OSDI 2022. Paper
9. Han et al., “Learning both Weights and Connections for Efficient Neural Networks,” NeurIPS 2015. arXiv:1506.02626
10. Mishra et al., “Accelerating Sparse Deep Neural Networks,” 2021. arXiv:2104.08378 (2:4 sparsity)

References (cont.)

11. Frantar & Alistarh, "SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot," ICML 2023. arXiv:2301.00774
12. Sun et al., "A Simple and Effective Pruning Approach for Large Language Models," ICLR 2024 (Wanda). arXiv:2306.11695
13. Verma, "Mastering LLM Techniques: Inference Optimization," NVIDIA Developer Blog, 17 Nov. 2023 (updated 15 Aug. 2024). Article
14. Wang, Spindler, "Model Quantization: Concepts, Methods, and Why It Matters," NVIDIA Developer Blog, 24 Nov. 2025. Article
15. Zandieh, Daliri, Hadian, Mirrokni, "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate," 2025. arXiv:2504.19874
16. Zandieh, Mirrokni, "TurboQuant: Redefining AI efficiency with extreme compression," Google Research Blog, 24 Mar. 2026. Article
17. Khattab et al., *LLM Class*: Lecture 1—Introduction and Course Overview (slide deck). PDF
18. Harvard University IACS, *CS AC215—Advanced Practical Data Science* (Spring 2024), Lecture 11: *Compression techniques* (slides 8–30 used for distillation figures). PDF
19. CMU 15-779 (*Lecture 17: LLM fine-tuning* — slide deck). PDF